Lecture 18 : Project Estimation Techniques (Contd.)

Introduction

In the dynamic world of software project management, accurately estimating project effort, cost, and duration is paramount to success. Without reliable estimates, projects can easily run over budget, miss deadlines, and fail to meet stakeholder expectations. This lecture delves deeper into various project estimation techniques, building upon previous discussions to provide a comprehensive understanding of how to approach this critical task. We will explore the fundamental differences between top-down and bottom-up approaches, examine the role of algorithmic models, and discuss the importance of calibrating these models to ensure their relevance and accuracy.

Effective estimation is not merely about predicting numbers; it's about understanding the underlying complexities of a project, managing risks, and making informed decisions. By mastering these techniques, project managers can enhance their ability to plan, execute, and deliver software projects efficiently. This session will equip you with the knowledge to select appropriate estimation methods, understand their strengths and weaknesses, and apply them practically to real-world scenarios, ultimately contributing to more predictable and successful project outcomes.

Core Concepts

Term	Definition	Significance
`Bottom-up Estimation`	A technique that identifies all individual tasks, estimates their costs, and then aggregates these estimates to determine the total project cost.	Essential when detailed historical data for similar projects is unavailable, providing granular control and potentially higher accuracy for well-defined tasks.
`Top-down Estimation`	A method that starts with an overall project estimate, often based on cost drivers and past project data, and then	Useful when past project data is available, offering a quicker, high-level estimate

Term	Definition	Significance
	distributes this estimate among various project components.	that accounts for system-level activities.
`Algorithmic Models`	Quantitative models that use mathematical formulas, often based on project characteristics like size, to estimate effort and duration.	Provides a structured, repeatable, and objective approach to estimation, allowing for sensitivity analysis and calibration.
`Software Size`	A measurable characteristic of software that correlates with effort, typically quantified using metrics like Source Lines of Code (SLOC) or Function Points (FP).	A fundamental input for algorithmic estimation models, as effort and duration are often derived from the estimated size of the software.
`Model Calibration`	The process of customizing a generic estimation model to fit specific project situations or environments, often by adjusting parameters based on historical data.	Increases the accuracy and relevance of general models by adapting them to unique organizational contexts and project characteristics.
`Simplistic Model`	A basic estimation formula where estimated effort equals system size divided by productivity (e.g., `Effort = Size / Productivity`).	Serves as a foundational concept but has limitations, as it oversimplifies complex project dynamics and may not account for various influencing factors.
`Productivity`	The rate at which work is completed, often measured in terms of output per unit of effort (e.g., lines of code per staff-month).	A critical factor in effort estimation, as it directly influences the calculated effort required to complete a project of a given size.

Detailed Analysis

1. Top-Down and Bottom-Up Estimation Techniques

Project estimation techniques can be broadly categorized into top-down and bottom-up approaches, each with distinct methodologies, advantages, and disadvantages.

Bottom-Up Estimation:
This approach is characterized by its granular detail and is particularly useful when there is no historical data about similar past projects.
- **Steps:**
1. **Break down project activities:** The project is meticulously broken down into smaller and smaller components. This decomposition continues until the tasks are small enough to be performed by one person within a short timeframe, typically one to two weeks.
2. **Estimate lowest-level activities:** Costs are estimated for each of these smallest, lowest-level activities.
3. **Aggregate estimates:** At each higher level of the project hierarchy, estimates are calculated by summing the estimates from the lower levels.
- **Advantages:**
* Allows the software group to estimate in a traditional, detailed fashion.
* Each group estimates components for which it has adequate experience, leading to potentially more accurate individual task estimates.
* More stable, as estimation errors in various components tend to balance out.
- **Disadvantages:**
* Can be very time-consuming due to the detailed breakdown required.
* May overlook many system-level costs such as integration, configuration management, and quality assurance, as these are not easily broken down into individual tasks.
* Can be inaccurate in early project phases when necessary information for detailed task breakdown is not yet available.

Top-Down Estimation:
In contrast, the top-down approach begins with a high-level view and is suitable when past project data is available.
- **Steps:**
1. **Produce overall estimate:** An overall estimate for the entire project is generated using effort drivers or cost drivers, often based on historical data from similar past projects.
2. **Distribute proportions:** This overall estimate is then distributed proportionally among the various components or phases of the project.
- **Example 1: Project Duration**

* Suppose an overall project is estimated to take 100 days.
* This total duration is then allocated to major phases:
* Design: 30% (30 days)
* Coding: 30% (30 days)
* Testing: 40% (40 days)
- **Example 2: Project Cost**
* If the total project cost is estimated at 500,000.
* This cost is distributed among components:
* Design: 20% (100,000)
* Programming: 30% (150,000)
* Testing: 40% (200,000)
* Documentation: 5% (25,000)
* Producing CDs: 5% (25,000)
* Further breakdown can occur, e.g., Design into D1 and D2, each taking 10% of the total, or Programming into GUI, database, and normal code with varying percentages.
- **Advantages:**
* Accounts for system-level activities like integration, documentation, and configuration management, which bottom-up methods might ignore.
* Requires minimal project details, making it faster and easier to implement, especially in early project stages.
- **Disadvantages:**
* Often fails to consider difficult low-level problems, tending to underestimate and overlook the complexities of individual components.
* Provides no detailed basis for justifying specific decisions or estimates, making it harder to defend the allocated proportions.

2. Algorithmic Models for Project Estimation

Algorithmic models provide a quantitative and structured approach to project estimation, particularly for effort and duration. These models rely on measurable project characteristics that correlate with the desired estimates.

- **Criteria for a Good Algorithmic Model:**
- **Defined:** Clear and unambiguous procedures.
- **Accurate:** Produces reliable estimates.
- **Objective:** Minimizes subjective factors.
- **Understandable:** Results are easy to interpret.
- **Stable:** Valid for a wide range of parameter values.
- **Easy to Use:** Practical for project managers.

- **Causal:** Does not require future data, relying on existing information.
- **Role of Software Size:**
- Directly estimating effort or duration from a problem description is challenging.
- Instead, effort and duration are often measured in terms of "size," a characteristic of software that is easily measured and correlates with effort.
- Common metrics for software size include Source Lines of Code (SLOC) and Function Points (FP).
- **Examples and Limitations:**
- COCOMO (Constructive Cost Model) and Function Points are prominent examples of algorithmic models.
- A common issue with `LOC`-based models like COCOMO is that they often start with a "guess" for size, then apply an algorithm to estimate. Ideally, estimation should begin with system characteristics, then apply the algorithm.

```
# Problem with LOC-based models:
# Guess -> Algorithm -> Estimate
# Desired approach:
# System Characteristic -> Algorithm -> Estimate
```

- **Advantages of Algorithmic Methods:**
- Generates repeatable estimations, ensuring consistency.
- Easy to modify input data, refine, and customize formulas.
- Efficient and supports a family of estimations or sensitivity analysis.
- Can be meaningfully calibrated to previous experience, enhancing accuracy.
- **Disadvantages of Algorithmic Methods:**
- Lacks the means to deal with exceptional conditions (e.g., exceptional teamwork, unique skill-task matches).
- Poor sizing inputs or inaccurate cost driver ratings lead to inaccurate estimations.
- Some factors, such as developer experience, are difficult to quantify for algorithmic input.

3. Model Calibration and Recommendations

Model Calibration:
Many estimation models are developed for specific situations and are inherently

calibrated to those contexts. However, to increase the accuracy of general models and make them useful outside their original environment, calibration is essential.
- **Purpose:** Customizing a generic model to fit specific organizational needs and project characteristics.
- **Calibratable Items:** Product types, operating environments, labor rates and factors, and relationships between functional cost items.

Recommendations for Project Estimation:
To improve the reliability and accuracy of project estimates, several best practices should be followed:
- **Avoid Single Estimates:** Do not rely on a single cost or schedule estimate, as it may lead to inaccurate results.
- **Use Multiple Techniques:** Employ several estimating techniques or cost models.
- **Compare and Analyze:** Compare the results from different techniques and investigate reasons for any significant variations. Ideally, different methods should yield similar results.
- **Document Assumptions:** Clearly document all assumptions made during the estimation process.
- **Monitor Assumptions:** Continuously monitor the project to detect when assumptions prove wrong, as this can jeopardize estimate accuracy.
- **Improve Software Process:** Enhance the overall software development process by maintaining a historical database of past projects, including details that can inform future estimations.

4. Simplistic Model and Practical Example

A basic, simplistic model for effort estimation is defined as:
`estimated effort = (system size) / productivity`

Where:
- `system size` could be in lines of code.
- `productivity` could be in lines of code per day or staff-month.

This model, while straightforward, has limitations as it oversimplifies the complex factors influencing project effort.

Example Calculation:
Consider a transaction project with 38,000 lines of code (38 KSLOC) and a productivity of 400 SLOC per staff-month (0.4 KSLOC/SM). We need to estimate the effort and shortest development time.

- **Estimated Effort:**

```
# Given:
# System Size = 38,000 SLOC = 38 KSLOC
# Productivity = 400 SLOC/staff month = 0.4 KSLOC/staff month

# Effort = (1 / Productivity) * System Size
# Effort = (1 / 0.4 KSLOC/staff month) * 38 KSLOC
# Effort = 2.5 staff month/KSLOC * 38 KSLOC
# Estimated Effort = 95 staff months (approximately 100 staff months as per lecture)
```

- **Shortest Time (Minimum Time):**
  The lecture uses a formula derived from empirical data, often seen in models like COCOMO, where time is related to effort by a power law.

```
# Using the calculation provided in the lecture slides:
# Min Time ≈ 1.875 * (Estimated Effort)^(1/3)
# Min Time ≈ 1.875 * (100)^(1/3)
# Min Time ≈ 1.875 * 4.64158883361
# Min Time ≈ 8.703 months (approximately 9 months)
```

This example demonstrates how a simplistic model, combined with empirical relationships, can be used to derive initial estimates for effort and duration.

Key Takeaways

- [ ] Project estimation techniques can be broadly classified into top-down and bottom-up approaches, each suited for different project contexts and data availability.
- [ ] Bottom-up estimation involves breaking down tasks, estimating at the lowest level, and aggregating upwards, ideal when detailed historical data is scarce.
- [ ] Top-down estimation starts with an overall estimate and distributes it, effective when past project data is available and for accounting for system-level activities.
- [ ] Algorithmic models use mathematical formulas and project characteristics (like size) to provide repeatable and objective estimates for effort and duration.
- [ ] Software size, measured by metrics like SLOC or Function Points, is a crucial input for algorithmic estimation models.

- [ ] Model calibration is vital for adapting generic estimation models to specific organizational environments, thereby increasing their accuracy and relevance.
- [ ] Relying on a single estimation technique is risky; using multiple methods and comparing their results helps identify potential inaccuracies.
- [ ] Documenting assumptions and continuously monitoring them throughout the project lifecycle is critical for maintaining estimate accuracy.
- [ ] Maintaining a historical database of past projects is essential for improving future estimation processes and calibrating models effectively.
- [ ] Simplistic models provide a basic framework (e.g., Effort = Size / Productivity) but often require more sophisticated adjustments to account for real-world complexities.

Conclusion

Understanding and applying various project estimation techniques is a cornerstone of effective software project management. This lecture has highlighted the contrasting yet complementary nature of top-down and bottom-up approaches, demonstrating how each can be leveraged depending on the project's stage and available data. We've also explored the structured world of algorithmic models, recognizing their power in providing objective and repeatable estimates, while also acknowledging their limitations, particularly in dealing with qualitative factors.

The emphasis on model calibration and the practical recommendations for estimation underscore a crucial message: estimation is not a one-time event but an iterative process that benefits from continuous refinement, historical data, and a critical evaluation of assumptions. By integrating these diverse techniques and best practices, project managers can move beyond mere guesswork, fostering a culture of data-driven decision-making. As we look ahead, the next lecture will delve into Function Points, another critical metric for software size, further enhancing our toolkit for robust project estimation.